TicketStar

Entrega 2: Infraestructura

Informe descriptivo y técnico · Programación Distribuida y Componentes · Proyecto Final

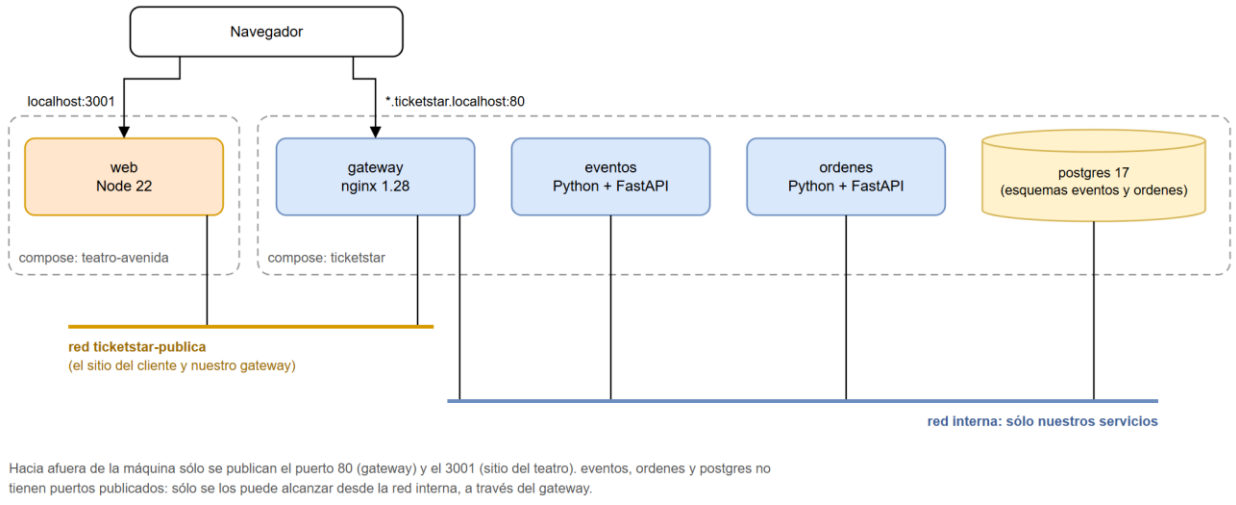
Grupo N°	[completar]
Integrantes	Francisco Aguiar, Rodrigo Rodriguez
Tag / Release	v2-infraestructura
Fecha de entrega	[completar]
Repositorio	https://github.com/franaguiargit/ticketstar

En esta entrega armamos el esqueleto real del sistema: cada parte corre en su propio contenedor, se puede prender y apagar por separado y ya se hablan entre ellas por la red. Todavía no se puede comprar una entrada de punta a punta, eso arranca en la Entrega 3, pero lo que está hecho anda de verdad y no es una maqueta.

1. Qué hay andando

Son cinco contenedores repartidos en dos sistemas separados, cada uno con su propio docker compose. Uno es TicketStar y el otro es el sitio de un cliente de prueba, el Teatro Avenida, que hace de "otra empresa".

Contenedor	Sistema	Qué hace	Hecho con
gateway	TicketStar	La única entrada desde afuera. Por el subdominio sabe de qué cliente es el pedido y lo manda al servicio que corresponde.	nginx 1.28
eventos	TicketStar	La cartelera de cada cliente: eventos y funciones.	Python, FastAPI
ordenes	TicketStar	Abre las sesiones de compra y muestra nuestra página de checkout.	Python, FastAPI
postgres	TicketStar	La base de datos. Cada servicio tiene su propio esquema y no toca las tablas del otro.	PostgreSQL 17
web	Teatro Avenida	El sitio de prueba del cliente: muestra la cartelera que le pide a TicketStar y abre la compra.	Node 22



Los dos compose, las dos redes y los únicos puertos que se publican.

[Pegar acá la captura de docker compose ps con los cuatro contenedores de TicketStar corriendo]

2. Modelo de ejecución: microservicios

Para nuestro sistema elegimos microservicios: cada parte es un servicio chico, con una sola responsabilidad, que corre en su propio contenedor. Entre el sistema del cliente y el nuestro, en cambio, la relación es cliente-servidor sobre HTTP: el teatro es el cliente que nos llama y nosotros somos el servidor que responde.

Por qué microservicios y no un solo servidor

- Cuando se abre la venta, casi toda la carga cae sobre órdenes. Si está separado, lo podemos escalar a él solo (en la Entrega 6 vamos a levantar varias instancias) sin tener que multiplicar todo el sistema.
- Si se cae un servicio, el resto sigue andando. Por ejemplo, si se cae la cartelera la gente igual puede terminar una compra que ya empezó. Lo probamos en esta entrega apagando eventos (sección 4).
- Cada servicio tiene sus propias tablas y se puede cambiar por dentro sin tocar a los otros. Eso lo vamos a formalizar con los contratos de la Entrega 4.

También tiene su costo: hay más cosas para levantar y más lugares donde puede fallar la red. Por eso arrancamos con pocos servicios y los vamos sumando a medida que hacen falta.

Por qué no P2P

En la venta de entradas tiene que haber alguien que decida quién se queda con cada butaca. Si cada nodo tuviera su copia del inventario y se pusieran de acuerdo entre ellos, sería mucho más difícil asegurar que una butaca no se venda dos veces. Necesitamos un lugar central que tenga la última palabra, y eso encaja mejor con servicios detrás de un gateway que con nodos iguales entre sí.

3. Cómo se conectan

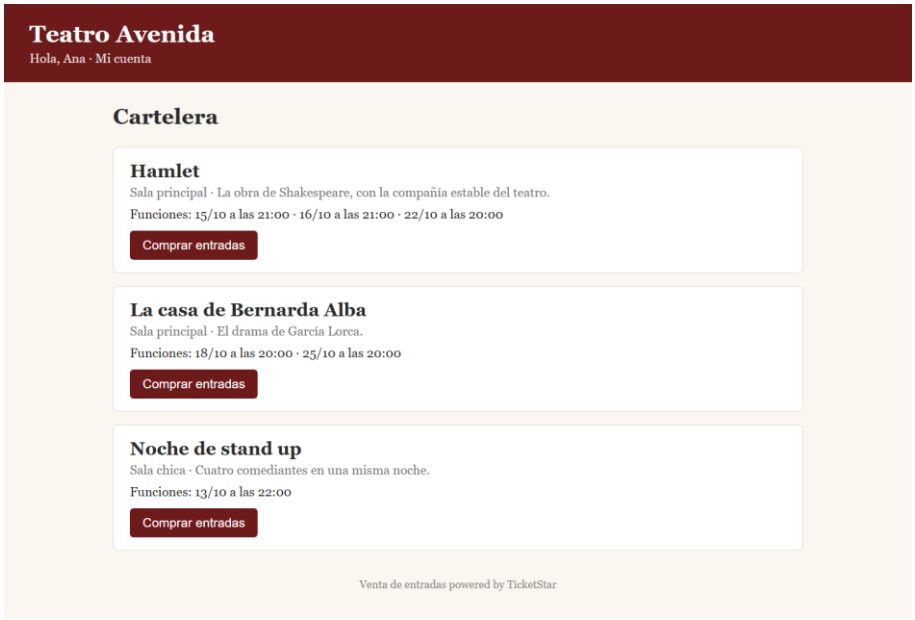
- El gateway es el único contenedor nuestro con un puerto publicado (el 80). eventos, ordenes y postgres no tienen puertos hacia afuera: solo se ven dentro de la red interna de Docker.
- Cada cliente entra por su subdominio: teatro-avenida.ticketstar.localhost y club-atletico.ticketstar.localhost. El gateway saca el cliente del subdominio y se lo pasa al servicio en un header. Si alguien manda ese header por su cuenta, el gateway lo pisa, así un cliente no puede hacerse pasar por otro.
- El sitio del teatro está en otro compose y solo comparte con nosotros una red, ticketstar-publica, donde además de él está únicamente el gateway. Desde el sitio del cliente se puede llegar al gateway pero no a nuestros servicios ni a la base, igual que pasaría con una empresa de verdad.
- El gateway busca cada servicio por nombre en cada pedido, con el DNS interno de Docker, y no una sola vez al arrancar. Lo hicimos así para que el gateway levante aunque algún servicio esté apagado, y para que lo vuelva a encontrar solo cuando se prende.

En desarrollo usamos subdominios de localhost en vez de ticketstar.com.ar, porque apuntan solos a la propia máquina y no hay que configurar nada. Así responde la API para cada cliente (recortamos un poco la salida):

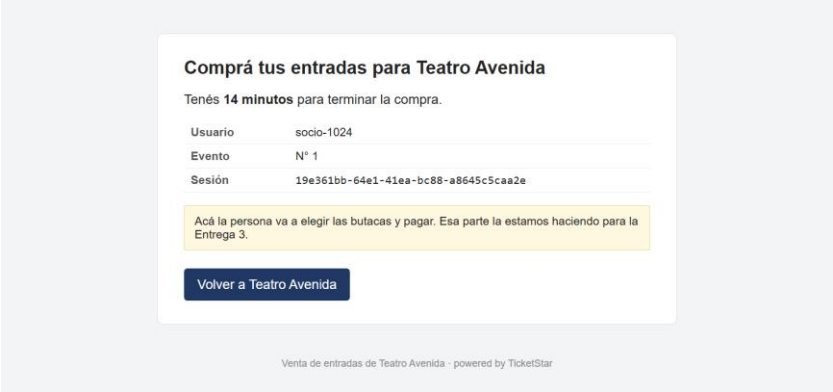
```
$ curl http://teatro-avenida.ticketstar.localhost/v1/eventos
{"cliente": "teatro-avenida", "eventos": [{"id": 1, "nombre": "Hamlet",
  "lugar": "Sala principal", "funciones": [...]}, ...]}

$ curl http://club-atletico.ticketstar.localhost/v1/eventos
{"cliente": "club-atletico", "eventos": [{"id": 4, "nombre": "Final del torneo",
  "lugar": "Estadio del club", "funciones": [...]}, ...]}
```

Y así se ve del lado del cliente. El sitio del teatro le pide la cartelera a TicketStar y, cuando la persona toca "Comprar entradas", abre una sesión de compra y la manda a nuestro checkout, en el subdominio del teatro:



El sitio del Teatro Avenida (Node) mostrando la cartelera que viene de TicketStar.

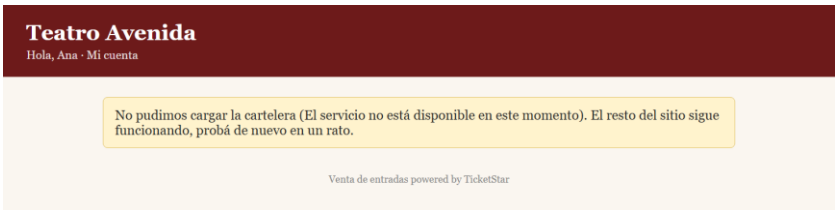


Nuestra página de checkout. Elegir butacas y pagar llega en la Entrega 3.

4. La prueba de que los nodos son independientes

Para mostrar que la separación es real, apagamos un nodo por vez con docker compose stop y miramos qué pasa con el resto. Para volver a prenderlo alcanza con docker compose start.

Qué apagamos	Qué pasa con el resto
eventos	La cartelera responde 503 con un mensaje claro. Las sesiones de compra siguen funcionando porque son de ordenes, y el sitio del teatro sigue arriba mostrando un aviso en lugar de la cartelera. Al prenderlo de nuevo, en unos segundos el gateway lo vuelve a encontrar.
postgres	eventos y ordenes siguen vivos: informan que la base está sin conexión y responden 503 en vez de un error 500. Cuando vuelve la base, vuelven a andar sin reiniciarlos.
gateway	El sitio del teatro sigue arriba y avisa que TicketStar no responde. Los servicios de adentro siguen corriendo; simplemente nadie de afuera los puede alcanzar.
web (el teatro)	Nuestro sistema sigue igual. La API le sigue respondiendo a cualquier otro cliente.



El sitio del teatro con el servicio de eventos apagado.

Para ver rápido qué está andando, el gateway tiene una ruta de estado por servicio:

```
$ curl http://localhost/estado/eventos
{"servicio": "eventos", "estado": "ok", "base": "ok", "instancia": "..."}
```

5. Cómo levantarlo

El paso a paso completo, con los requisitos y los problemas comunes, está en el README de la raíz del repo. En resumen, y después abrir http://localhost:3001:

```
docker compose up -d --build          # TicketStar
docker compose ps                     # tienen que aparecer los cuatro contenedores
cd clientes-demo/teatro-avenida
docker compose up -d --build          # el sitio del teatro, en otro compose
```

6. Qué cambió respecto de la Entrega 1

- Por ahora la página de checkout la sirve el servicio de órdenes. Si crece, la pasamos a un servicio aparte.
- Usamos un solo Postgres con un esquema por servicio, en vez de una base por servicio. Cada servicio sigue sin tocar las tablas de los otros, pero es un contenedor menos para levantar.
- Para simular que el cliente es otra empresa, el sitio del teatro corre en su propio compose y solo comparte con nosotros la red del gateway.
- Todavía no hay HTTPS ni permisos entre sistemas: cualquiera que llegue al gateway puede usar la API de un cliente. Eso es parte de la Entrega 5.

7. Qué sigue

- Entrega 3: la reserva de butacas con el caso de concurrencia y un script que lo demuestre, órdenes consultando a eventos por REST, y RabbitMQ con el servicio de notificaciones.
- Entrega 4: el contrato de la API en OpenAPI y el descubrimiento de servicios con Consul. En ese sprint también arrancamos el segundo cliente de prueba (club-atletico, con la forma B), como estaba planeado.